
CHAPTER 6

Experiments

This chapter describes the models implemented and the experiments performed to train and evaluate them. First, the baseline experiments will be explained. Then we will explain the iterative process with which different experiments have been performed in order to find an optimal prediction model and dataset.

6.1 Baseline forecasting models

A baseline model is a very simple, prototype model that serves as a reference to evaluate the performance of more complex models. In this case, the baseline model receives as input the draw, home-win and away-win odds from the bookmaker *Bet365*. Based on this input, the model will predict the outcome of the matches. Later, the results of more complex models will be compared with this model, which will be used to see if the new models perform as expected, because if a new model matches or exceeds the accuracy of the baseline, it will mean that it has a better predictive ability than the odds assigned by the bookmakers.

Various baseline models have been implemented based on different well-known Machine Learning algorithms, such as Support Vector Machine (SVM), Random Forest (RF) or Multi-Layer Perceptron (MLP). It is important to note that the odds have been transformed to probabilities (between 0 and 1), simply by doing the inverse of the odds.

Baseline I: max odds The first baseline is the simplest. It simply consists of the prediction made by *Bet365*, as the model chooses the maximum probability between draw, home-win and away-win. As mentioned in the odds analysis (Figure 5.10) bookmakers never predict that the match will end in a draw, as they always assign a higher probability to the victory of one of the teams. The success rate was 54.45%. Figure 6.1 shows the confusion matrix of the results and the AUC-ROC curve. It can be seen in both figures that this simple model has great difficulty in distinguishing draws, with this class having the smallest Area Under Curve (AUC) of the three 0.61.

Baseline II: SVM The second baseline consists of a Support Vector Machine (SVM) model trained with 3-fold Cross-Validation (CV) and the hyperparameters have been optimised with a randomised search of hyperparameters, using the class of *Scikit-Learn*, *RandomizedSearchCV*. This class already incorporates the CV training option. The optimised hyperparameters have been the SVM regularisation hyperparameter (C), the *kernel* used in the SVM (linear, polynomial or rbf), the hyperparameter *gamma*, which is the ker-

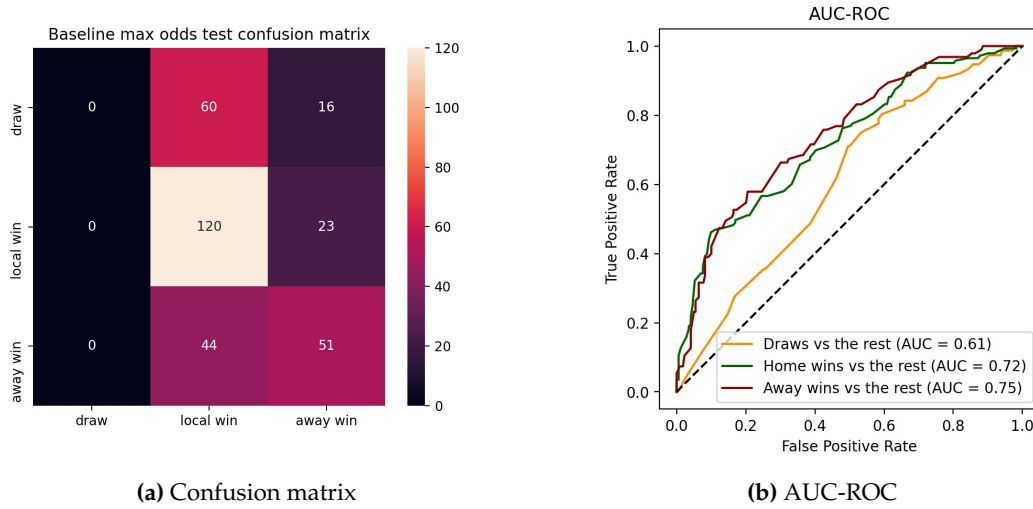


Figure 6.1: Confusion matrix and AUC-ROC curve of max-odds baseline.

nel coefficient for the RBF-SVM and polynomial SVM, and, finally, the hyperparameter *degree* indicating the degree of the polynomial in the polynomial kernel.

The best-performing model obtained a test score of 54.78% with $C = 2.99$, $kernel = rbf$ and $gamma = 0.75$.

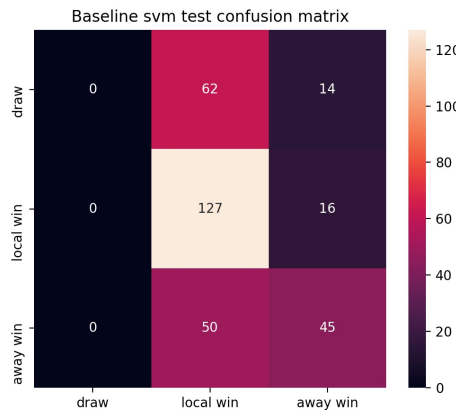


Figure 6.2: Test evaluation of Support Vector Machine based baseline confusion matrix.

Baseline III: Random Forest The baseline based on a Random Forest has also been trained with 3-fold CV, and the hyperparameters optimised with random search have been the number of estimators (decision trees), the maximum depth of the trees, the minimum number of samples needed to make a split in the tree, the minimum number of samples to create a leaf and the maximum number of leaves that the trees can have. All these parameters are optimised to avoid overfitting and to obtain the maximum possible success in the test.

The best-performing Random Forest model obtained a test success rate of 54.73%, with 112 decision trees, 96 minimum matches to create a split, 4 minimum samples per leaf and a maximum of 5 leaves per tree.

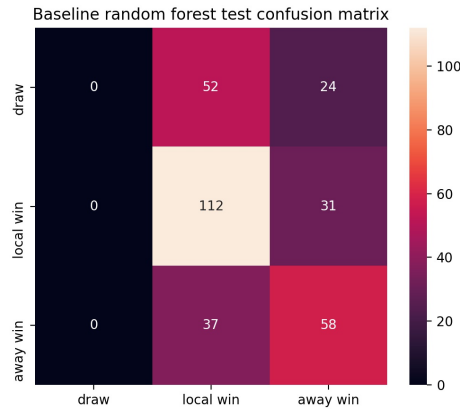


Figure 6.3: Test evaluation of Random Forest based baseline confusion matrix.

Baseline IV: MLP Finally, a baseline based on Multi-layer Perceptron with a hidden layer of three units has been created. Like the previous models, it has also been trained with 3-fold CV, and in this case no hyperparameter optimisation has been performed. The hyperparameters used were Cross Entropy as an error function, Stochastic Gradient Descend (SGD) as an optimiser with a learning rate of 0.1, 100 epochs and a batch-size of 32. A success rate of 54.78% was obtained, slightly higher than the other models, except for SVM, which equals it.

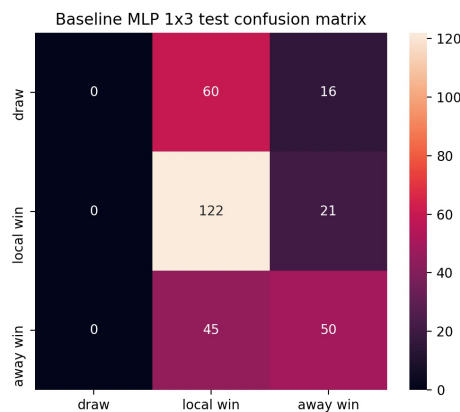


Figure 6.4: Test evaluation of Multi-Perceptron based baseline confusion matrix.

These test success rates will try to be outperformed by more complex Machine Learning models. According to the confusion matrices, all models behave very similarly, predicting far more local wins than the other possible outcomes, which makes sense given the distribution the results followed. However, it is remarkable to note that no model predicts any draws, possibly because of the unbalanced nature of the dataset (Table 5.9). AUC-ROC is shown in Figure 6.1, where we can see that the draws are the most difficult for bookmakers to identify. Therefore, it will also be the objective of the models to be developed to improve the accuracy in the prediction of draws.

	Max-Odds	SVM	Random Forest	MLP
%	54.46	54.78	54.73	54.78

Table 6.1: Baseline models test accuracy.

6.2 Forecasting models

This section explains the experiments carried out in which the created datasets, explained in the previous chapter, have been trained with two different Machine Learning algorithms: Neural Networks and Gradient Boosting. As explained previously in the Methodology, the approach taken was to start by carrying out experiments with simple solutions and models, and then build up the necessary software infrastructure to analyse the results and implement increasingly complex algorithms and solutions if the analysis so indicated.

We will now explain the experiments that have been carried out with simple MLP models and default configurations. We will start by explaining the experiments to debug the correct functioning of the infrastructure created to train the models.

6.2.1. Artificial Neural Networks experiments

The first Machine Learning algorithm that will be used to predict the results will be Neural Networks, specifically MLP networks. The aim of this part is to find the optimal neural network model to predict the results of football matches. During this part, therefore, we will perform a series of experiments in which neural networks with different configurations (architecture and hyperparameters) will be trained. The different configurations will be decided according to a heuristic based on the analysis of the previous experiments (Figure 1.1).

The neural network designed will be a MLP that will receive as input in the first layer the features of a football match, and will obtain three values in the output, one for each class: home win, away win or draw. The three output values will be normalised with a softmax in order to represent the probability that the sample belongs to each class. More precisely, the output of the softmax would be an estimate of the probability that the target (home win, away win or draw) is y given a match x , i.e. an estimate of $p(y|x)$.

It is true that the softmax output of the output layer of an ANN does not exactly represent a probability, mainly because although ANNs often make good predictions it is not accurate to use this output as true probabilities. In practice, the output of ANNs does not produce calibrated probabilities, i.e. they do not represent the probability that a prediction is correct, but contain a bias that tends to predict higher probabilities by being overconfident in the class with higher probability. This phenomenon is called overconfidence [Guo et al., 2017].

However, although [Guo et al., 2017] presents a solution for the calibration of the output of an ANN, this problem is outside the scope of this work, so we will assume the output of the softmax function as probabilities that have a margin of error. In addition, an error analysis of these probabilities will be performed using a confidence interval.

We have talked about the interpretation of the ANN output as probabilities, however it is essential to talk about the metrics the model uses to learn from the error it makes

during training. The networks we will train will use Cross Entropy (CE), the function normally used for training models in multiclass classification problems. For an n match we seek to minimise the CE function, which looks like this:

$$L_n = - \sum_{c=1}^C w_{(c)} t_{(n,c)} \log p_{(n,c)} \quad (6.1)$$

being $p_{(n,c)}$ the output softmax function for the match n and class c (draw, local or away win):

$$p_{(n,c)} = \frac{\exp x_{(n,c)}}{\sum_{j=1}^C \exp x_{(n,j)}} \quad (6.2)$$

The target values against which the model compares its prediction to measure its error are represented in a One-Hot Encoder vector where the index of the correct class has a 1 while the others have a 0. Our One-Hot Encoder vector has a length of three values, one for each class (home win, away win and draw), with draw being the first position of the vector (index 0), second position being home win (index 1) and away win the last position (index 2). The labels of each class of each match (i.e. each One-Hot Encoder index of each match) are represented as $t_{(n,c)}$ in Eq. 6.1. In this way, only the error in the target class is penalised, since the other labels of the One-Hot Encoder are set to 0 and vanish the error in those outputs. How to improve the model's evaluation of the training error will be discussed later.

Coming back to the estimation of the probabilities, these are integrated in the Cross-Entropy implementation provided by the PyTorch library (which we have used), so in the output of the model we have not used any softmax and we directly pass the value returned by the output layer to the Cross-Entropy function, which by the way uses a Log-softmax, which provides advantages such as being computationally cheaper and penalises the error more; and on the other hand, we use the output value of the network to calculate with softmax the probabilities of each class and then select the class with a maximum probability as prediction. The class that we finally predict $\hat{t}$ is calculated with the following formula:

$$\hat{t}_n = \arg \max_{p_{(n,c)}} \quad (6.3)$$

Finally, the accuracy is obtained by calculating the average success rate, i.e., if $\hat{t} = t$, of all the matches. On the other hand, at first, the model uses the ADAM algorithm as an optimiser with a default configuration with a learning rate (lr) of 0.001. Furthermore, no normalisation is applied to the ANN layers, and the weights of the layers are initialised by default. The activation function used will be ReLU. The data, coming from the Event Based Dataset, have a dimensionality of 20 features. Before training, the data will be split into two random splits, one for training and one for testing, with an initial ratio of 75-25. In addition, the data will be normalised before being fed into the ANN. The network will train for e epochs, which will be a value to be optimised, and the network will be optimised, i.e. its weights will be updated for each batch of data, i.e. each b samples. The size of each batch (b) is another hyperparameter to optimise, but initially this will have a default value of 32 samples per batch. The model will be trained with the training set using 5-folder Cross-Validation to validate the model using in each iteration a part of the data to validate. Finally the model will be re-trained with the whole training set and will be validated by making inference from the testing set. It is important to clarify a seed will be assigned to the whole pipeline to ensure the reproducibility of the training.

The different experiments carried out with this family of algorithms will be detailed below:

Bug-Free Models In order to carry out the experiments, a software infrastructure had to be implemented to run the pipeline of experiments. The pipeline serves to have the same platform on which to train all the models (from the simplest to the most complex), to log information about the training and to save both these logs and the results. Once the basic infrastructure has been implemented and following the proposed methodology (Figure 1.1), a series of experiments have been carried out to debug this implementation and ensure that there are no errors or bugs in the training and in the log.

In order to test the correct functioning of the training pipeline, we have used a simple MLP with an architecture of 1 input layer with 10 neurons, no hidden intermediate layer and a multi-output of 3 neurons, one for each class to be predicted. With this, the neural network is intended to predict the probability of a match resulting in a home win, away win or draw. The input will be 20 dimensions, one for each feature of each record (each match). In total this network has 263 parameters to train. The other hyperparameters are the default ones, we will use ADAM as optimiser, Cross-Entropy as loss-function, ReLU as activation function and we will not use regularisation of any kind.

The test experiment consists of overfitting a training batch (8 samples, but it could be even less), and once we reach an error of 0 or very close to it, check the alignment of the predictions and the targets, and look at the logs to see if they make sense and work correctly.

If we look at the learning curve, we can see, on the one hand, how the model has over-trained the batch to the maximum, achieving an error of 0 and an accuracy of 100% in the train, while in the test the error has been increasing iteration by iteration. This is due to the fact that the model has learned the patterns that follow the train data from memory, without having any capacity to generalise when new samples are introduced. Another observation that can be made from this test is that the accuracy obtained in an

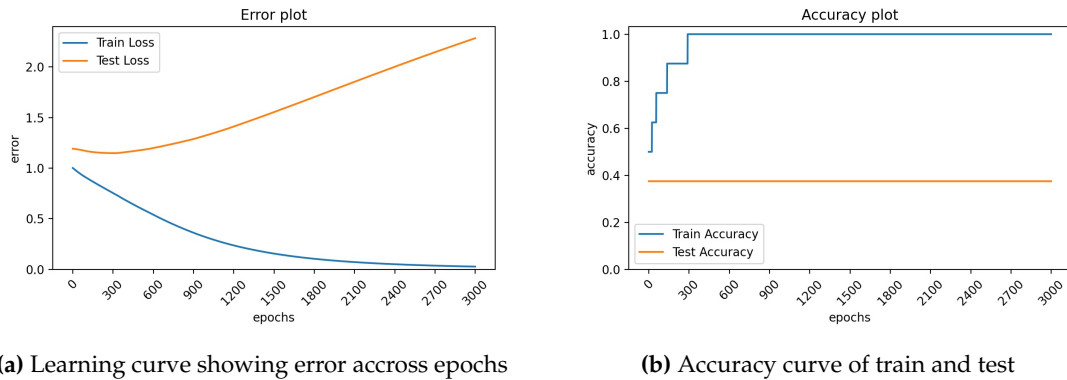


Figure 6.5: Event based data batch overfitted with a 263 parameter MLP.

experiment can be maximised with a non-optimal error. As can be seen in Figure 6.5, an accuracy of 100% is reached in the train at epoch 500, however, at that instant the error was still quite high, close to 0.8. This gives an indication that model error is not the only indicator to look at when analysing the results and performance of a model, and that it is therefore not necessary to obtain the minimum error to achieve good or even optimal accuracy. Figure A.1 shows the evolution of the weights during the test that has been carried out to check the correct working of the infrastructure on which the experiments will be carried out. It can be seen how, from the beginning of the training, most of the parameters converge, at different rates, towards the final weight distribution.

The experiments carried out with the entire dataset and applying the methodology proposed above are described below. These first experiments have been carried out with the Event Based dataset.

Event Based Dataset Experiments The first experiments have been carried out to get an idea of how different architectures behave with this data, i.e. what results are obtained with ANNs of different depth and amplitude (neurons per layer). These experiments have been performed with the same configuration as the validation of the experiment infrastructure in the previous section, i.e. with a default configuration. The difference is that in this section we use all the samples of the dataset and the training batch that is passed to the network is 32 records. The network has been optimised with ADAM (default parameters), with a learning rate of 0.001, CE has been used as loss-function and has been validated with 5-folder CV, as explained at the beginning of this section (6.2.1). Furthermore, the trainings will have a duration of 250 epochs, meaning that the training dataset will be passed to the network 250 times. This value is an initial value that will have to be adequate according to the results, but, a priori, it has been chosen because we consider it appropriate given the limited number of samples we have.

The results of the architecture testing experiments are shown in Table 6.2. In this table it can be seen that for networks with more complexity, i.e. with a higher number of parameters, the model learns the training data better, although with considerable overfitting. A model with 10,000 parameters achieves a success rate of 90% in the training set, but only 40% in the test. The more complex models, which have greater depth and more parameters, are probably too complex for the data we have, both in terms of the number of records (very limited) and the dimensionality of the input (20 dimensions). Another interesting point, which we have already mentioned above, is the fact that the model with the lowest test error does not coincide with the model with the highest test accuracy. This phenomenon will be analysed in more detail later. On the other hand, it is the simpler models with deeper networks (10-20 layers) that best generalise the test data, however, this error does not translate into good accuracy and they only achieve a 45% accuracy in the test, compared to 48% for the medium models of 2000 and parameters with only 3 layers. It is important to mention that when we talk about n layers, we do not include the output layer but we do include the input layer, so in an n layer model we would have $n - 1$ hidden layers in addition to the input and output layers.

params. \ layers	Train Error				Test Error				Train Accuracy				Test Accuracy			
	3	4	10	20	3	4	10	20	3	4	10	20	3	4	10	20
500	0.89	0.89	1.07	1.06	1.33	1.32	1.07	1.07	0.59	0.6	0.45	0.45	0.33	0.43	0.43	0.43
750	0.85	0.86	0.86	0.89	1.39	1.39	1.32	1.31	0.62	0.61	0.63	0.67	0.45	0.39	0.3	0.28
1000	0.85	0.87	0.9	0.91	1.37	1.36	1.3	1.31	0.61	0.6	0.64	0.69	0.44	0.31	0.24	0.25
1500	0.83	0.82	0.9	0.9	1.38	1.44	1.29	1.33	0.64	0.63	0.65	0.66	0.32	0.44	0.38	0.35
2000	0.8	0.8	0.89	0.85	1.4	1.44	1.3	1.40	0.65	0.65	0.61	0.59	0.48	0.43	0.43	0.40
3000	0.76	0.74	0.75	0.72	1.5	1.55	1.71	1.88	0.69	0.7	0.66	0.58	0.43	0.4	0.33	0.35
5000	0.67	0.68	0.75	0.7	1.57	1.64	1.82	1.07	0.73	0.72	0.67	0.46	0.48	0.37	0.31	0.42
10000	0.52	0.37	0.7	0.62	1.78	2.49	1.82	2.15	0.82	0.9	0.66	0.74	0.38	0.4	0.42	0.43

Table 6.2: Experiments results of different ANN architectures with default parameters on the Event Based dataset.

Figure 6.6 shows a comparison between four of the models shown in Table 6.2. These four models are detailed in Table 6.3. The comparison shows that the more complex models with more parameters learn faster and also have more flexibility in modelling the training data. However, these more complex models (3000 and 5000 parameters) have a considerably higher test error, which means that these models overtrain the training data. A contradiction, mentioned above, can be seen in the test accuracy graph where the

models with the highest accuracy are also those with the highest error.

Modelo	Parameters	Layers	units perm layer
500	443	3	10-15-3
750	743	4	20-10-5-3
3000	3023	4	50-30-10-3
5000	4943	3	70-45-3

Table 6.3

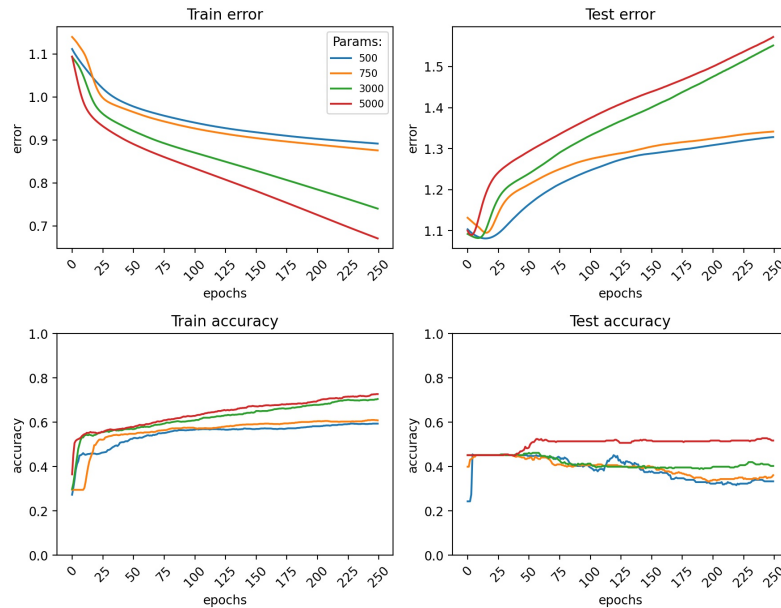


Figure 6.6: Error and accuracy learning curves for the 500, 700, 3000 and 5000 parameter ANNs.

Therefore, on the one hand, we have models that suffer from underfitting, i.e. the network is not flexible enough to model the training data and therefore converges in training with a high error. This may be due to different non-exclusive factors:

- **The model is too simple:** It may be that the model is too simple for the complexity of the data. In that case you could approach the solution by training a model with more complexity, either with more layers or more neurons in the layers. You could also reduce the complexity of the data by reducing its dimensionality. This is done by applying some dimensionality reduction technique.
- **Hyperparameters optimization:** these models have default settings, so you could modify the hyperparameters of the model to improve performance, for example, you could increase the Learning Rate to make bigger "jumps" when Gradient Descent updates the weights or increase the number of training epochs.
- **Speed of learning:** learning is rather slow in these models. Hyperparameters can be optimised to improve model performance, but a number of methods can also be applied to optimise model performance and make the model more stable and robust. Some of these methods are Batch normalisation and weight initialisation.

On the other hand, we have models that suffer mostly from the opposite, overfitting, i.e. the model learns the training data well but does not know how to generalise to new

data not seen during training. Let's say that the model memorises the training data and matches the parameters to these data to the millimetre. This is the case for the 3000 and 5000 parameter models in Figure 6.6. In this case, in addition to optimising the hyperparameters such as the learning rate or the epochs, other solutions could also be applied:

- **Add more data:** this is the preferable option. It is widely accepted that more data, and more quality data, is the most effective way to reduce overfitting. However, we do not have more data in this dataset, so other solutions will be considered first before experimenting with a new dataset.
- **Regularization:** it would be useful to apply some method to regularise the ANN so that it learns to generalise better. Some of the techniques that can be applied are weight decay, drop out or applying weights to the error function. The methods used will be explained.

The first solutions that have been applied have been aimed at reducing overfitting and improving the performance of our ANN to converge to an optimal solution more quickly. These solutions have been **Batch Normalization (BN)**, **Weight decay** and **Drop-out regularization**, as well as changing certain hyperparameters in a manual and intuitive way to see what impact they have on the model.

In order to apply these methods in the experiments, the code had to be modified and extended to support the new functionalities without changing the way of training and saving the results and logs. For the other changes, however, no code modification was required, as weight decay is a parameter of the ANN optimiser.

The first method, Batch Normalization, is used to train ANN in a faster and more robust way by normalising (re-centring and re-scaling) the inputs of the network layers. As explained in a previous chapter, BN corrects the Internal Covariate Shift (ICS) of the input of each layer during training and unifies them under the same distribution, thus reducing the interdependence between layers. In addition, extra-parameters are trained in the BN layer to adjust the input distribution and exploit the non-linearity threshold of the activation function. It is important that the input of the activation function is at the non-linearity threshold to reduce the probability of Vanishing and Exploding Gradient problems occurring and for the ANN to converge faster to its optimal solution. This is why the BN is usually applied to the input of the activation function. This allows a larger learning rate to be used as the network is now more stable. In addition, BN reduces overfitting because by centring and rescaling the inputs we are applying noise, this noise makes it difficult for the network to "memorise" the training data, especially the smaller the size of each batch. We can see that BN does not have the same impact in all cases. In more complex networks, as in this 3000 parameter network, it has more effect and we see how the learning converges faster and is more stable. However, in simpler networks it does not have the expected effect and we see how the error and accuracy is more similar to training without BN.

However, BN has not managed to reduce overfitting, in fact, it has increased it. To analyse the degree of underfitting and overfitting of a model we use bias-variance decomposition, which as its name suggests decomposes the model error into three parts, an irreducible error, bias and variance, the latter two being related to underfitting and overfitting respectively. The irreducible error would be the minimum possible error (whether human or not). In our case we will consider it as 0 since we have a way of knowing this value at the moment, and therefore we will assume that it is possible to achieve an error of 0. On

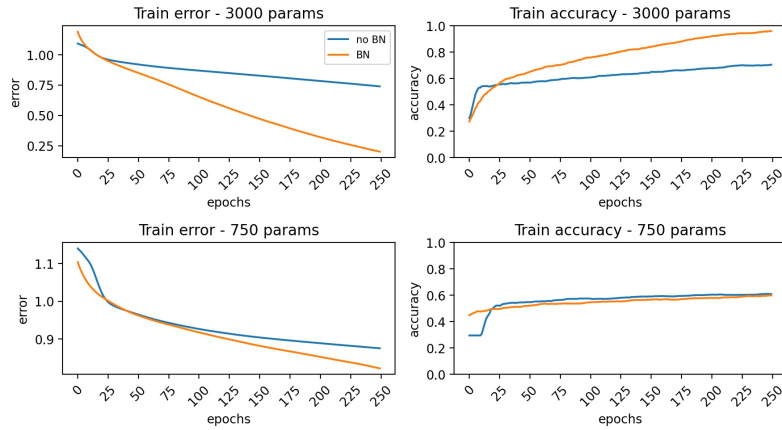


Figure 6.7: Effect of Batch Normalization on training for a 750 and a 3000 parameters model.

the other hand, the bias is the training error and the variance is the difference between the error in the test and the train error.

Bias is related to underfitting because both are related to erroneous assumptions of the model during training, ignoring relevant relationships in the data. Variance and overfitting are both related to the sensitivity of the model to variations in the data, with the model having an excessive sensitivity (since it models even noise) in the training data that penalises it for new test data. When applying the bias-variance decomposition we assume that both data sets (train and test) come from the same distribution [Neal et al., 2018].

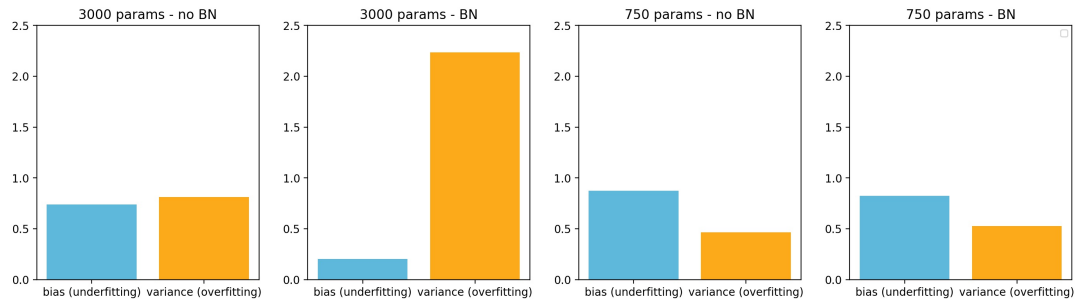


Figure 6.8: Bias-variance decomposition of trained non-BN and BN Neural Networks.

Figure 6.8 shows that BN reduces bias (underfitting) especially when the model has enough flexibility to model the data (~ 3000 parameters), while when the model has few parameters (~ 750) it has little room for improvement and flexibility. On the other hand, we see how BN does not reduce overfitting at all and, in fact, increases it by a factor of three.

We have seen so far that we need moderately complex models if we want to have a model with the ability to fit the data correctly. As we mentioned before, increasing the complexity of the ANN helps to reduce the error in training. However, we must now also reduce the overfitting so that our model has a better test accuracy.

To do this we have first performed several experiments with different weight decay values. Weight decay is a regularisation technique in which we apply an L2 penalty to the weights in the error function of our ANN in order to smooth them. Normally an ANN

that is overtrained has weights with very large values (positive or negative) that allow it to be very flexible when modelling all the training data. The idea of weight decay is to penalise during backpropagation the fact of having weights with very high values, forcing the ANN to look for an optimal smoothie configuration.

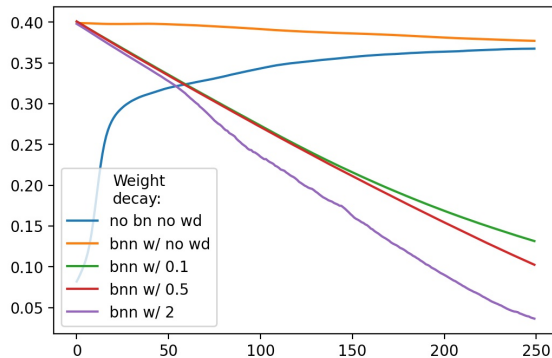
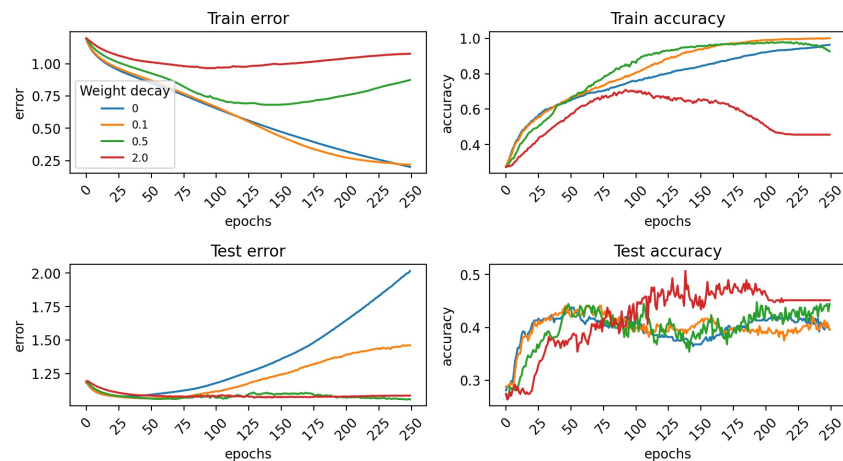


Figure 6.9: Mean of input activation function of a hidden layer during the training for 4 different models: Vanilla ANN without BN or weight decay, ANN with BNN but no weight decay and three ANN with BNN and weight decay of 0.1, 0.5 and 2 respectively.

Figure 6.10: Error and accuracy for train and test experiments on weight decay.



In Figure 6.9 we can see how the larger the weight decay penalty is, the faster the values of the activation function input tend to 0. This is because in these models the weights tend towards 0 due to the L2 penalty applied. We also see how the model with BN and the one without BN converge to the same point, however, the one with BN is already initialised at a closer value so it takes less time to converge. This is another way of looking at the efficiency of training with BN versus without BN.

However, the application of weight decay to reduce overfitting has not produced the desired results (Figure 6.10). The truth is that although it does its job of regularising and has led to better test results, it seems that the penalty when updating the weights is too strong and does not allow the ANN to model the data well, especially in terms of accuracy.

We have also applied another well-known regularisation technique called Drop-out. This technique consists in temporarily deactivating some nodes of the ANN (input and hidden layers), thus creating a new architecture from the original ANN. The nodes are deactivated with a probability p . When an ANN seeks to minimise its error function, it does so by taking into account all the neurons at once, so during training, complex correlations are created between neurons where some "solve the error of others", which occurs intensively during overfitting. The Drop-out technique solves the problem of overfitting because by randomly eliminating certain neurons in each iteration we are

creating a new architecture in each iteration (maintaining the weights of the neurons that are active) and therefore the neurons are forced to have less interdependence, as they are updated each time they are in a different architecture [Srivastava et al., 2014]. Actually, Drop-out is a variant of bagging with several models. It is also important to mention that, according to the related literature, the use of Drop-out with Batch Normalization is discouraged [Goodfellow et al., 2016].

Figure 6.11: Error and accuracy for train and test experiments on drop-out.

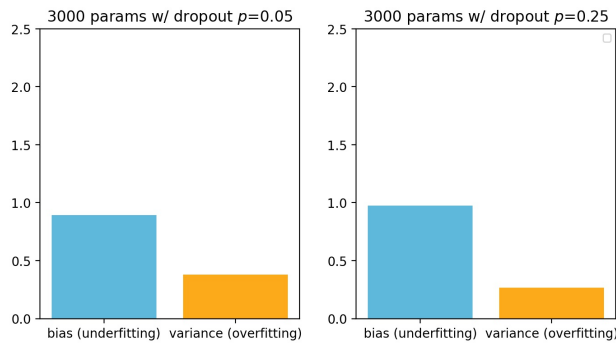
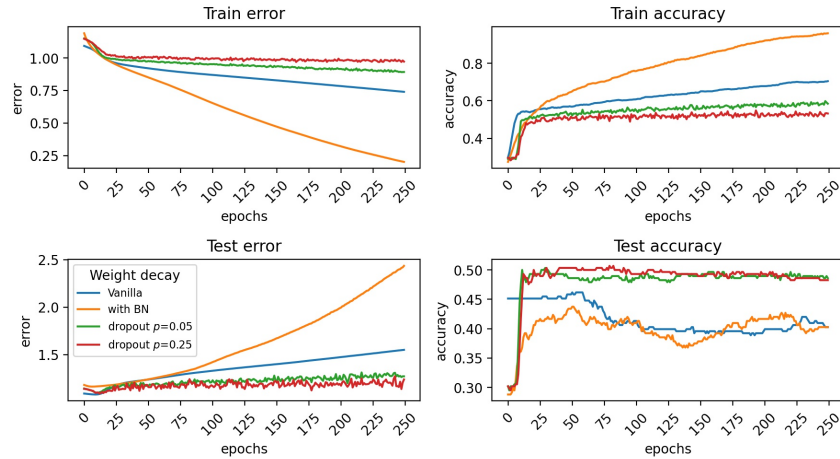


Figure 6.12: Bias-variance decomposition of trained non-BN and BN 3000 parameters Neural Networks.

We have performed several experiments with two values of p : 0.05 and 0.25. As it can be seen in Figure 6.11, this technique also accelerates the learning of the model, we can see how the accuracy increases enormously in the first 25 epochs. Furthermore, in this case we see improvements in the accuracy compared to the experiments with Weight decay, in which the learning was irregular and erratic, here it is much more regular, faster and also the model manages to reach a success rate of 50% in the test. The decrease in overfitting in the Drop-out experiments (Figure 6.12) compared to the BN experiments (Figure 6.8) is also significant.

Up to now, several techniques have been shown to correct for underfitting and overfitting in training. However, these exploratory experiments can still be improved as they have been performed independently and with many hyperparameters kept as defaults. These experiments have been useful to get an idea of what impact these techniques can have on our model and how it reacts to the addition of new functionalities. In order to further optimise our model, we have now performed a search for the best hyperparameters for this dataset.

To perform this search for optimal hyperparameters, we have used the Random Search technique, which consists of defining a limited space of hyperparameter values and ran-

domly choosing values from this space. This is a good method for discovering unintuitive hyperparameter combinations and values that with Grid Search would require more search time (Figure 6.13). With Grid Search it would take a long time to cover all values and we could be ignoring a value that minimises or maximises the objective function. With Random Search it is more likely to cover more values in the space.

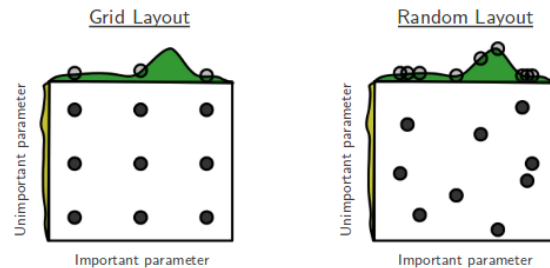


Figure 6.13: Difference between Grid Search and Random Search. The green distribution plot shows how a parameter maximize an objective function.

In this first Random Search experiment we seek to optimise the hyperparameters (Figure 6.4) of the ANN using the functionalities we have mentioned so far: BN, weight decay and Drop-out. We cannot show, let alone explain, every single experiment done during

Hyperparameter	Search space	Data type	Constraints
Type of net	Vanilla ANN, BN ANN or Drop-out ANN	categorical	
Units per layer	[3-100]	integer	
Layers	2, 3, 4, 9	integer	non-including output layer
Drop-out p	(0, 0.2]	float	only in drop-out
Optimizer	ADAM or Stochastic Gradient Descend (SGD)	categorical	
Momentum	[0-1]	float	only in SGD
Weight decay	[0-1]	float	1/2 choices will be 0
Nesterov	True or False	Boolean	>0 momentum and 0 dampening
Dampening	[0, 0.5]	float	
Learning rate	0.00001, 0.0001, 0.001, 0.01, 0.1	float	
Betas	([0, 1.0], [0, 1.0])	tuple	only in Adam
Scaler	Normalizer, MinMax, MaxAbs, Standarize	categorical	
Batch size	16, 32, 64, 128	categorical	

Table 6.4: Hyperparameters to optimize with Random Search.

Random Search. However, we can summarise the execution of all the experiments by aggregating different statistics of the results obtained. Therefore, we are going to talk about the hyperparameters (Figure 6.4) that have the most impact on the performance of the model.

In Figure 6.14 we can see the hyperparameters with the best test accuracy. We can see that practically all experiments are around 45% correct in the test. However, there are a few experiments that reach and pass the 50th percentile. It is the Drop-Out models that generally achieve the best results, although the Vanilla ANN (dumb model class) also obtains good results, far behind are the BN models. Of the Drop-out models, the best values for p seem to be clearly between 0.1 and 0.2. On the other hand the number of layers of the model is quite important and better results have been obtained with ANNs with few hidden layers (2 and 3). Another important hyperparameter is the learning rate, which although all its values behave similarly with respect to the accuracy, it is true that a very small LR makes it more difficult for the ANN to learn, as we can see that for an LR of $1e - 5$ practically all the experiments have the same accuracy. The weight decay does not obtain good results, except with values of practically 0. The rest of the hyperparameters

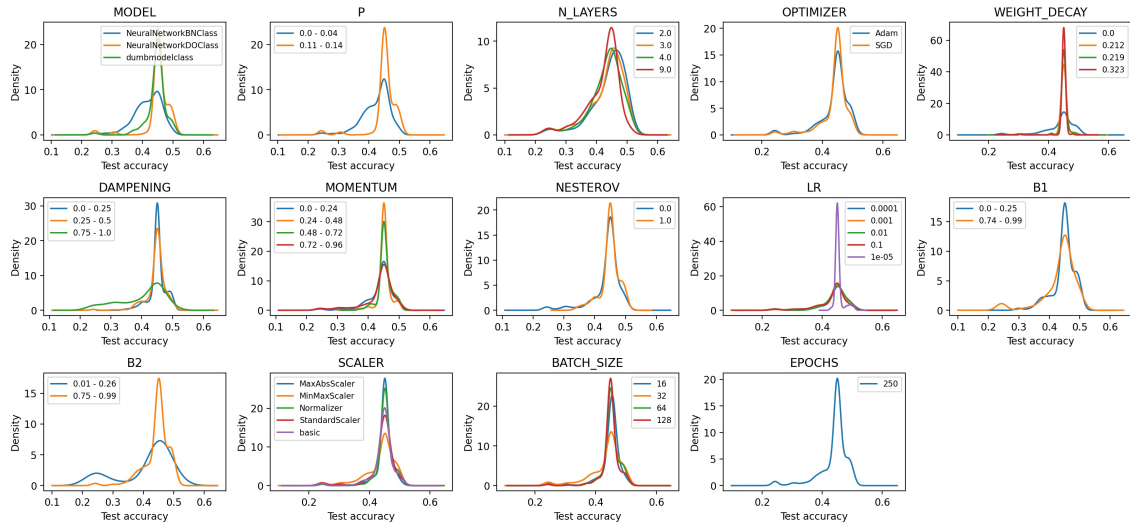


Figure 6.14: Test error distribution for each hyperparameter and value.

do not seem to significantly condition the accuracy of the model.

Model	Architecture	Optimizer	Lr.	Train err	Train acc	Test. err	Test acc	F1 draws	F1 home	F1 away
Drop-out	73-53-21-3	SGD	0.1	0.947	0.546	1.194	0.512	0.061	0.622	0.487
Drop-out	53-21-3	SGD	0.001	0.989	0.518	1.171	0.510	0.077	0.630	0.444
Drop-out	68-4-7-3	SGD	0.1	0.947	0.546	1.195	0.508	0.031	0.622	0.483
Drop-out	34-11-56	Adam	0.00001	0.992	0.512	1.123	0.508	0	0.619	0.509
Drop-out	42-18-18	Adam	0.001	0.934	0.544	1.201	0.508	0.264	0.621	0.428

Table 6.5: Top 5 best configurations on the Random Search.

We have so far analysed how hyperparameters impact model performance and which values are optimal. We have also applied solutions to reduce underfitting and overfitting in order to get the best test results. If we check which are the best results we see that the best results have little overfitting, in fact the top 5 are trained with Drop-out (Table 6.5). However, there is one aspect that we have not yet analysed and that is, which matches the model gets right and which ones it misses, and what is the reason for this.

F1-score is a very appropriate metric to evaluate the results of a model trained with an unbalanced dataset, and as we have already mentioned in a previous chapter, our dataset is not balanced (Table 5.9). The F1-score is a metric that tells us the relationship between the Recall and Precision of a model. The accuracy of a model indicates whether what you have predicted is correct, i.e. the ratio of True Positives to all Positives (True + False). On the other hand, the Recall indicates whether you have predicted the samples of a class correctly, i.e. the ratio of True Positives among all the samples of that class (True Positives + False Negatives). F1-score is the harmonic mean of both metrics. In conclusion, F1-score indicates whether the model knows how to differentiate correctly between the different classes.

We can see in Table 6.5 that despite having a high accuracy in general (>50%) the F1-score in the draws is almost 0 or directly 0. Let's compare the difference between the model with the highest accuracy and the fifth with the highest accuracy, which is also the one with the highest F1-score. We can see in Figure 6.15 how the first model, despite having the best overall accuracy, is really bad at predicting draws. The distribution that the predictions follow does not match the actual values at all (Table ??). Only 1% of the predictions are draws compared to 24% of the actual values. We can see that when it

doubts between home win or draw, it always goes for the first one, because there is no case of home win that it has predicted as a draw. In the second model, we see that it considerably improves the draws and increases to 11% of the total predictions. However, this model is still far from 24% of draws.

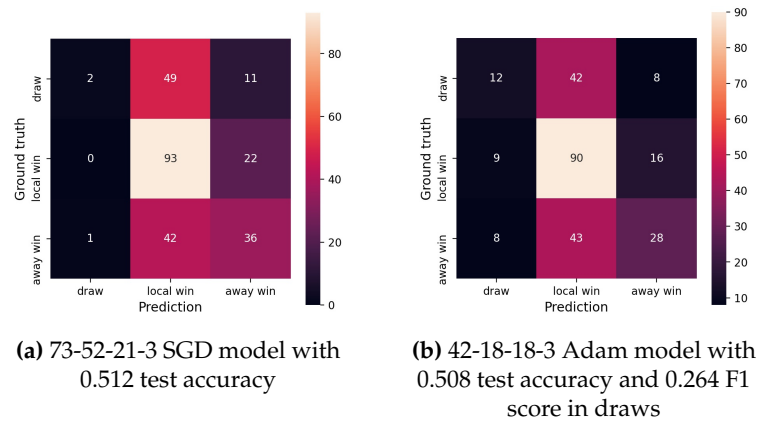


Figure 6.15: Confusion matrix for two ANN with different accuracy in draws.

(%)	Draws	Home wins	Away wins
Ground truth	24	45	31
Predictions model 1	1	72	27
Predictions model 2	11	68	21

Table 6.6: Distribution in percentage (%) of each class in the test set labels and in the test predictions for models 1 (73-52-21-3) and 2 (42-18-18-3).

We can see that the models have problems when it comes to identifying draws (Recall), as they only predict at most 20% of these. The models under-predict draws, so solutions must be found to ensure that the model predicts draws better without penalising home and away wins.

For this reason, two solutions have been proposed. The first is related to simplifying the training for the model, and consists of applying a series of dimensionality reduction or feature selection techniques to the data. We will do a series of experiments applying Variance Threshold and ANOVA to select the best features, PCA to transform the data into a reduced dimensional space and we will do some experiments with a series of features selected manually after the analysis of the data (Table 5.15).

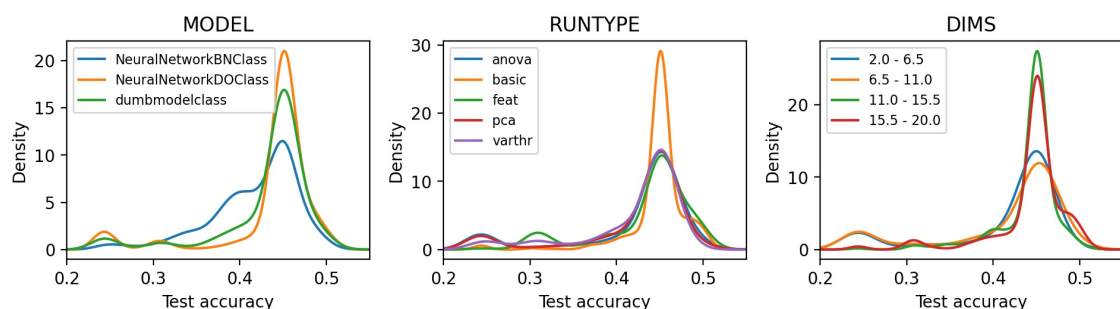


Figure 6.16: Test error distribution for each hyperparameter and value in feature selection experiments.

Method \ Dimensionality	5	10	12	15	20
All features	-	-	-	-	0.512
Manual selection	-	-	0.507	-	-
Variance Threshold	0.457	0.473	-	0.49	-
ANOVA	0.487	0.5	-	0.5	-
PCA	0.512	0.510	-	0.516	-

Table 6.7: Results of feature selection experiments. This table shows the higher accuracy reached for each method employed and new dimensional space of data.

Figure 6.16 and Table 6.7 show how the model tends to make better predictions with inputs of larger dimensionalities as it provides more information to the model. However, the methods used to select the best features do not all obtain the same results. PCA obtains the best results in all the different dimensions to which the input data has been reduced. Especially with 15 dimensions we obtain the best accuracy of this experiment. Very close is the accuracy obtained training with the 20 features (all available features). We also mention the good result obtained with the manually selected features after the data analysis, which validates in a certain way the analysis made in a previous chapter about the importance of the variables. Finally, the worst results obtained with the method of selecting the features that exceed v variance threshold, Variance Threshold.

In addition, in order to try to balance the distribution of predictions to that of the labels, we have carried out a series of experiments focused on finding the optimal weights assigned to each class in the error function. When calculating the error during forward-propagation, each class is weighted by its weight in order to penalise some classes more than others. In this case, a higher weight is assigned to the draws so that it penalises the error in the draws more and learns more from the error it commits. This is intended to make the model more familiar with the draw and more sensitive to it. These weights have also been optimised with Random Search, optimising the other hyperparameters as well (Table 6.4).

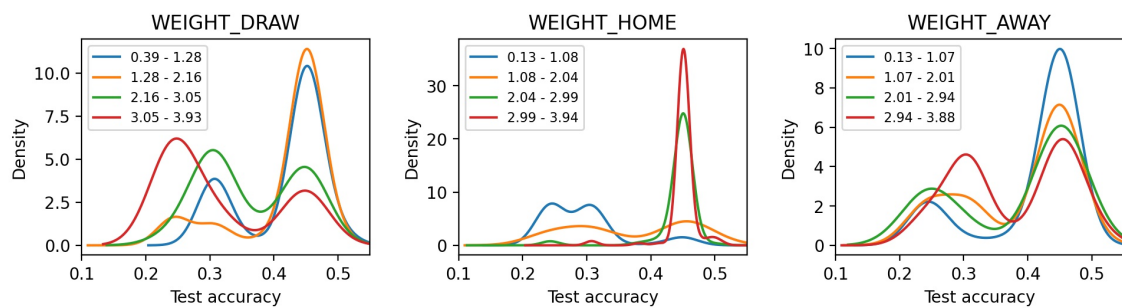


Figure 6.17: Test error distribution for each class loss-weight.

We can see in Figure 6.17 how these parameters have a great impact on the accuracy of the model. However, the relationship between them is complex because they interact with each other. When we assign a weight to a class, it not only influences that class but also the training of the other classes. Remember that the labels we pass to the CE loss (our error function) are One-Hot Encoded vectors and therefore in each match only the error in the target class is penalised even though the ANN gives as output a probability to each class. In other words, if the match is a home win, only the error in the output of

the home win will be penalised, but the probability of a draw or away win will not be penalised and the ANN will not learn from it. With the default setting (weights of 1 for each class) all classes are penalised equally. However, by having an unbalanced dataset, in practice, the error in the output of local wins will be penalised on many occasions, and conversely the output of away wins and draws on very few occasions. This causes the model to learn more in one direction than in others, i.e. it is biased by the large number of home wins. If you also add that draws are usually very tight matches and that, as there are fewer correlations of the features with the result, there is less separability in the dimensional space of the features, it makes it very difficult for the models to learn and guess the draws.

With the aim of rebalancing the error, different combinations of weights have been tested for each class. We see in Figure 6.17 that high values for the draw penalty cause a significant drop in the overall accuracy of the model ($\sim 25\%$). This is because in this case we are forcing large changes in its weights when it commits a draw miss, biasing the model's learning towards over-predicting draws. Figure 6.18 shows the difference in gradients with different values of loss-weights. Therefore, the value of these hyperparameters also affects the performance of the model in terms of robustness and stability of learning. When we say that there are strong interactions between the three hyperparameters, it is not only the absolute value of the error weight that matters but the ratio or difference between the three (Figure 6.19). If we apply three small ratios but one class has half as much penalty as the others, it will be penalised half as much as the others and therefore the model will potentially predict fewer matches of that class.

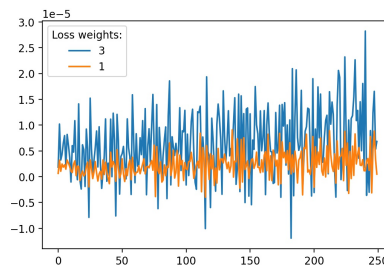


Figure 6.18: Input layer gradients during training with loss-weights of 1 and 3 for all the classes.

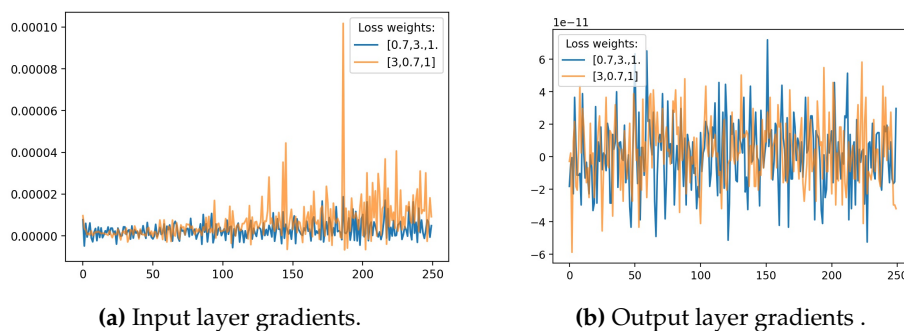


Figure 6.19: Loss-weights focusing in home wins (blue) and draws (orange).

Finally, the weights that have given the best results in improving the prediction distribution have been those around 1. By using a slightly higher weight for draws than for home and away wins, the model also improves the prediction of draws (Table 6.8), but slightly reducing the overall accuracy of the model (Figure 6.20).

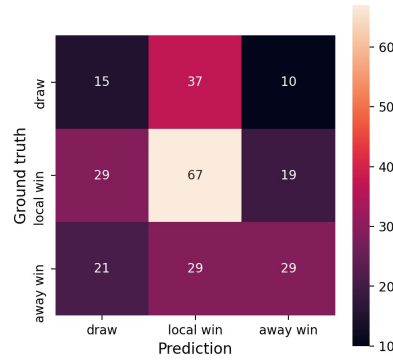


Figure 6.20: Confusion matrix of a model with the following loss weights: draw: 1.2, home win: 0.8 and away win: 1.0.

(%)	Draws	Home wins	Away wins
Ground truth	24	45	31
Loss weights: 1.2 - 0.8 - 1.0	25	52	23
Loss weights: 1.1 - 0.9 - 1.0	6	54	40
Loss weights: 1.2 - 0.7 - 1.1	24	34	42

Table 6.8: Distribution in percentage (%) of each class in the test set for different loss weight values.

So far we have applied solutions to reduce overfitting and underfitting, solutions to improve the performance of the model so that it learns faster and more stably in training, and we have also applied solutions to make the model make better and more varied predictions. We have improved the results from the first experiments where the models were 48% correct, whereas now we have reached 51.6% correct. We have also improved the accuracy of draws, where the first models had an F1-score of approximately 0, i.e. almost ignoring draws. We have now achieved an F1-score on ties of 0.26 and have managed to fit the distribution of predictions to the distribution of labels. However, we believe that we can obtain even better results, both in terms of accuracy and class separability.

Given the limitations offered by this dataset in terms of number of records to train and test, we have created a new dataset (5.2.2) with new features and 10 times more records than the one used so far. With more data and therefore variability, we expect the model to be able to learn new patterns and relationships in the data.

Historical Dataset Experiments El propósito de los siguientes experimentos es mejorar los resultados de nuestros anteriores experimentos en base a entrenar nuestro modelo con un dataset más grande y con otro tipo de información. Como hemos mencionado en un capítulo anterior, el Historical Dataset contiene datos acerca del estado de forma a corto y largo plazo de los dos equipos que juegan un partido. Con esto, se busca que el modelo tenga no sólo información de los últimos n partidos sino también de meses y años atrás. En el anterior modelo podíamos tener el caso de un equipo, como el Chelsea o el Bayern Munchen, que viniese de una mala racha y que el modelo automáticamente le diera como ganador al equipo rival ya que sólo se fija en los últimos n partidos. Ahora el modelo tomará también en consideración los últimos meses y años de ambos equipos.

Como también hemos mencionado anteriormente, tenemos varias versiones de este dataset, las cuales compararemos. Para ello, como en los experimentos con el Event-Based dataset, realizaremos una Random Search para encontrar los hiperparámetros óptimos de los